

Lab

Inheritance

Lab: Inheritance

Objective

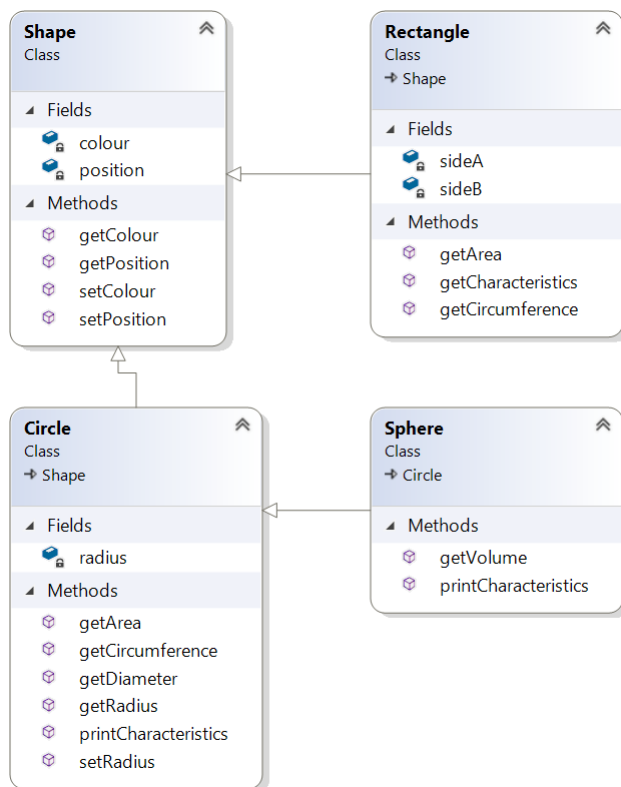
The main goal of this lab is to help you develop the ability to create new types by extending existing ones, while also incorporating specialised functionality to meet specific requirements.

Overview

This lab introduces the foundational concepts of inheritance. As discussed in the accompanying session, implementing inheritance requires an initial class that defines the core behaviour or functionality to be extended.

Step-by-step

1. Open the Labs project and then add a new package called **lab7**.
2. Write a class called **Program** with a `main()` method in the **lab7** package.
3. Create the following class structure:



4. Create a constructor for **Shape** to set its colour and position.
5. As you can see, **Circle** extends **Shape** and **Sphere** extends **Circle**.

a) **Position** is of type **Point** which is a class with built-in x and y.

Tip: `import java.awt.Point`

b) **Colour** is of type **java.awt.Color**

You will use **Math.PI** to get the value of **PI** which you'll use to calculate the area and circumference of a circle.

Volume of a sphere is calculated as $\frac{4}{3} * \text{PI} * R * R * R$

You can use the **Math.pow(R, 3)** function or $R * R * R$

6. Create getters and setters for each field (colour, radius, etc.) as indicated in the class diagram above.
7. The **getCharacteristics()** method returns a **String** containing all the attributes of the shape. It will be up to the caller how to display this information.
8. Create different kinds of shapes in **main()** like Rectangle, Circle, and Sphere.
9. Print the characteristics of these objects.
10. Create an **ArrayList<Shape>** called **shapes** in the **main()**.
11. Add the shapes which you created earlier into the **shapes ArrayList**.
12. Create an enhanced **for** loop to scroll through each shape and print its **colour** and **position** (x and y).

How does this work? How can we store a shape, like a Rectangle, in a list of Shape objects? This will be answered in the next chapter, where you'll discover how object-oriented principles make this possible.



QA.com